

TraceGraph – Phase 3 Prototype Progress Report

CECS 551 Term Project

1. Prototype Overview

TraceGraph is a graph-based retrieval and question answering prototype designed for multi-hop reasoning over long-form textual data. The main objective of the system is to move beyond traditional flat retrieval methods by constructing a sparse graph of document chunks and using that graph to discover related evidence paths. This allows the prototype to retrieve bridge information more effectively and support answers that require combining evidence from multiple locations in a corpus.

The system takes a collection of input documents, splits them into manageable chunks, enriches those chunks with structural and semantic metadata, and then builds relationships among them through graph edges. During question answering, the system retrieves relevant seed chunks, traverses connected chunks in the graph, assembles the resulting evidence into a compact context, and produces an answer with supporting citations and retrieval-path explanations.

2. Implemented Functionality

At this stage, the prototype already supports the main workflow required for a working proof of concept. The data pipeline includes document ingestion, chunking, and preprocessing. Text is divided into chunks so that each part of the source material can be treated as a graph node. The preprocessing stage also enriches chunks with entities and keywords, which later help during graph construction and retrieval.

The graph construction pipeline has also been implemented. Each chunk is stored as a node, and multiple edge types are created to represent relationships among nodes. These include sequential links between adjacent chunks, shared-entity links for chunks mentioning similar entities, lexical similarity links for semantically related content, and optional relation-based edges when additional associations are detected.

The retrieval subsystem is functional as well. It supports seed retrieval using BM25, entity overlap, and optional embedding-based similarity. Instead of depending on one single retrieval score, the system combines multiple retrieval signals and ranks chunks using weighted score fusion. This allows the prototype to identify promising starting evidence even for questions requiring indirect or cross-document reasoning.

Graph traversal is also implemented. After identifying strong seed nodes, the system expands outward through neighboring chunks while respecting configurable depth and budget limits. This step helps the prototype capture bridge evidence that might not appear in the initial top-ranked chunks but is still necessary to answer the question correctly.

In addition, an agent-style answering workflow has been developed. The agent drafts an answer from the assembled context, checks whether evidence appears incomplete, and can optionally refine the query and launch another retrieval cycle. This creates a revisitable question answering loop rather than a single one-shot response pipeline.

3. Current System Flow

The current working system can be summarized in the following stages:

Stage	Description
Input	The system loads a source corpus or a prepared demo dataset for experimentation.
Preparation	Documents are chunked, cleaned, and enriched with entities and keywords.
Graph Building	Each chunk becomes a node, and semantic or structural relationships are added as edges.
Seed Retrieval	BM25, entity matching, and optional embeddings are used to rank initial evidence chunks.
Traversal	The retriever expands from seed nodes to neighboring nodes that may contain bridge evidence.
Context Assembly	Retrieved evidence is merged into a compact context within a token budget.
Answering	The agent synthesizes an answer and attaches supporting citations or retrieval path information.
Evaluation	The output can be assessed using metrics such as Exact Match, F1, latency, and retrieval coverage proxies.

4. Prototype Status

The current Phase 3 system can be considered a working proof of concept and an MVP-level research prototype. The core pipeline is already operational from end to end, including data preparation, graph construction, retrieval, traversal, answer generation, evaluation, and output reporting. This means the prototype already demonstrates the technical feasibility of the proposed idea.

Several major components are complete. The retrieval and graph traversal logic are implemented and configurable. The callback-based agent loop has been added to support iterative retrieval when evidence is missing. Persistence and output folders are available for saving graphs, chunks, retrieval bundles, evaluation summaries, and run artifacts. The repository also includes test coverage for chunking, graph construction, traversal, metrics, citation formatting, persistence, and command-line behavior.

At the same time, a few refinements remain open for the final phase. Relation extraction can be improved so that graph edges represent deeper semantic connections. Evidence grounding can also be made more fine-grained at the sentence level rather than only at the chunk level. In addition, a stronger language-model-backed answer synthesis module can be integrated to improve answer quality on harder reasoning tasks.

5. Challenges and Next Steps

One challenge in the current prototype is that graph quality depends heavily on how well chunks are enriched and linked. If edges are too sparse, the system may miss important evidence paths. If edges are too loose, traversal may introduce noise into the final context. Balancing these trade-offs is an important area for further tuning.

Another challenge is that retrieval performance is easier to control than final answer quality. Even when useful evidence is retrieved, the synthesis step may still need improvements in evidence ranking, context compression, and answer grounding. This becomes especially important for multi-hop questions where evidence is distributed across several chunks.

The next step is to extend the prototype into a stronger final version by improving relation extraction, refining traversal heuristics, tuning score fusion weights, and integrating more advanced answer generation. Additional benchmark experiments, ablation studies, and case-study examples will also be included to strengthen the final evaluation.

6. Summary

Overall, the Phase 3 implementation demonstrates that the core TraceGraph idea is technically feasible and already operational as a working prototype. The project includes an end-to-end graph-based retrieval pipeline, a revisitable question answering workflow, and a supporting evaluation framework. The prototype is modular, testable, and well-positioned for further refinement in the final phase.